

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: Основы алгоритмизации и программирования

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
на тему

ИГРОВОЕ ПРОГРАММНОЕ СРЕДСТВО РИТМИЧЕСКОГО
ВЗАИМОДЕЙСТВИЯ С СЕТЕВЫМ РЕЙТИНГОМ

БГУИР КР 6-05-0612-01 106 ПЗ

Студент: гр. 451001 Држевецкий Н. А.

Руководитель:
асс. Шамына А.Ю.

Минск 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 АНАЛИЗ ПРОТОТИПОВ, ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ПРОЕКТИРУЕМОМУ ПРОГРАММНОМУ СРЕДСТВУ	7
1.1 Примеры решения аналогичных задач	7
1.2 Требования к проектируемому программному средству.....	7
2 АНАЛИЗ ТРЕБОВАНИЙ К ПС И РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ	10
2.1 Описание функциональности ПС.....	10
2.2 Спецификация функциональных требований	12
3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА	15
3.1 Разработка алгоритма ПС	15
3.2 Алгоритмы отдельных модулей	15
4 СОЗДАНИЕ (КОНСТРУИРОВАНИЕ) ПРОГРАММНОГО СРЕДСТВА	21
4.1 Структура программного средства	21
4.2 Описание логики, процедур и функций.....	22
5 ТЕСТИРОВАНИЕ, ПРОВЕРКА РАБОТОСПОСОБНОСТИ И АНАЛИЗ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ	28
5.1 Тестирование программного средства.....	28
5.2 Экспериментальная проверка.....	29
5.3 Анализ полученных данных	30
5.4 Результаты тестирования.....	30
6 РУКОВОДСТВО ПО УСТАНОВКЕ И ИСПОЛЬЗОВАНИЮ	32
6.1 Установка приложения “BeatKey”	32
6.2 Эксплуатация приложения “BeatKey”	33
ЗАКЛЮЧЕНИЕ	36
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	37
ПРИЛОЖЕНИЯ.....	38
ВЕДОМОСТЬ	61

ВВЕДЕНИЕ

Современные информационные технологии активно применяются при разработке интерактивных программных средств, в том числе игровых приложений, использующих сетевое взаимодействие и удалённое хранение данных. Особое место среди таких приложений занимают ритм-игры, сочетающие обработку мультимедиа, работу в режиме реального времени и элементы сетевого обмена информацией. Подобные программные средства позволяют не только организовать игровой процесс, но и реализовать взаимодействие пользователей через удалённые серверные сервисы и базы данных.

Ритм-игры представляют собой жанр компьютерных игр, в котором игровой процесс напрямую синхронизирован с музыкальным сопровождением. Пользователь должен выполнять определённые действия в строго заданные моменты времени, соответствующие ритму музыкальной композиции. Примерами популярных игр данного жанра являются *osu!*[1], *Friday Night Funkin'*[2] и *A Dance of Fire and Ice*[3]. Подобные проекты демонстрируют эффективность сочетания визуальной обратной связи, аудиосинхронизации и обработки пользовательского ввода в режиме реального времени.

В современных игровых приложениях всё чаще используются сетевые технологии, позволяющие хранить пользовательские данные на удалённых серверах, синхронизировать результаты между устройствами и формировать онлайн-таблицы лидеров. Использование клиент-серверного взаимодействия позволяет расширить функциональность локального программного средства и обеспечить возможность хранения и получения данных через сеть Интернет.

Целью данной курсовой работы является разработка игрового программного средства «BeatKey» с поддержкой сетевой таблицы лидеров. Приложение представляет собой ритм-игру для персонального компьютера, ориентированную на выполнение нажатий клавиш в такт музыкальному сопровождению, с возможностью загрузки пользовательских музыкальных композиций и хранения результатов игровых в удалённой базе данных.

Программное средство реализовано на языке программирования C++ с использованием фреймворка Qt. Для реализации сетевого взаимодействия используется модуль Qt Network, обеспечивающий выполнение HTTP-запросов и обмен данными с удалённой базой данных Firebase Realtime Database[4]. Использование Qt позволяет объединить в одном приложении работу с графическим интерфейсом, мультимедиа, пользовательским вводом и сетевыми технологиями.

В ходе выполнения курсовой работы были поставлены следующие задачи:

1 Анализ существующих ритм-игр и сетевых механизмов хранения результатов пользователей.

2 Проектирование архитектуры приложения и структуры клиент-серверного взаимодействия.

3 Реализация игрового процесса и механики подсчета очков.

4 Разработка системы загрузки и отображения таблицы лидеров через сеть Интернет.

5 Реализация обмена данными с удалённой базой данных с использованием HTTP-запросов и формата JSON.

6 Разработка пользовательского интерфейса и системы визуальной обратной связи.

7 Тестирование работоспособности программного средства и корректности сетевого взаимодействия.

8 Подготовка документации, описывающей структуру, алгоритмы и результаты разработки.

Разработка данного программного средства представляет собой практическую задачу по созданию интерактивного сетевого приложения, сочетающего обработку графики, аудио, пользовательского ввода и сетевого обмена данными. Результатом курсовой работы является полностью функционирующее игровое приложение с поддержкой удалённой таблицы лидеров, соответствующее поставленным требованиям и пригодное для дальнейшего расширения и развития.

1 АНАЛИЗ ПРОТОТИПОВ, ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ПРОЕКТИРУЕМОМУ ПРОГРАММНОМУ СРЕДСТВУ

1.1 Примеры решения аналогичных задач

На сегодняшний день существует ряд программных продуктов и игр, реализующих механику ритмического взаимодействия пользователя с музыкальными композициями. Рассмотрим наиболее известные из них.

osu! — популярная ритм-игра, в которой игроку необходимо нажимать на игровые объекты в строго определённые моменты времени, синхронизированные с музыкальным сопровождением. Игра поддерживает загрузку пользовательских карт, содержащих тайминги нажатий, и предлагает различные режимы взаимодействия.

Достоинства:

- 1 Высокая точность синхронизации с музыкой.
- 2 Поддержка пользовательских карт и музыкальных файлов.
- 3 Большое сообщество и разнообразие контента.
- 4 Развитие реакции и чувства ритма.

Недостатки:

- 1 Высокий порог входа для новых пользователей.
- 2 Сложный интерфейс и система настроек.
- 3 Зависимость от интернет-сервисов для полноценного функционала.

Friday Night Funkin' — ритм-игра, в которой игрок должен нажимать клавиши в такт музыке, ориентируясь на визуальные подсказки в виде движущихся стрелок. Игра ориентирована на реакцию и запоминание ритмических паттернов.

Достоинства:

- 1 Простой и понятный игровой процесс
- 2 Низкий порог входа.
- 3 Яркая визуальная обратная связь.
- 4 Развитие координации и внимания.

Недостатки:

- 1 Ограниченные возможности по созданию собственных карт.
- 2 Отсутствие встроенного редактора таймингов.
- 3 Жёсткая привязка визуальных элементов к игровому стилю.

1.2 Требования к проектируемому программному средству

1.2.1 Назначение разработки.

Разрабатываемое программное средство BeatKey представляет собой ритм-игру с поддержкой сетевой таблицы лидеров, предназначенную для взаимодействия пользователя с музыкальными композициями посредством нажатия клавиш в заданные моменты времени. Цель разработки — создание игрового приложения, позволяющего загружать музыкальные файлы и карты таймингов, подсчитывать результаты пользователя и передавать их в удалённую базу данных для формирования сетевого рейтинга игроков.

1.2.2 Состав выполняемых функций.

- 1 Загрузка музыкальных файлов формата MP3 и WAV.
- 2 Загрузка карт таймингов из текстовых файлов.
- 3 Воспроизведение музыкальных композиций.
- 4 Отображение игровых объектов, синхронизированных с музыкой.
- 5 Обработка пользовательского ввода в режиме реального времени.
- 6 Подсчет очков и количества здоровья (НР).
- 7 Отображение текущего результата и игровых уведомлений.
- 8 Поддержка пользовательских настроек (громкость, бинды клавиш).
- 9 Сохранение результатов игроков в удалённой базе данных.
- 10 Загрузка и отображение сетевой таблицы лидеров.
- 11 Обмен данными с сервером посредством HTTP-запросов.

1.2.3 Входные данные.

- 1 Музыкальные файлы.
- 2 Файлы карт таймингов.
- 3 Нажатия клавиш пользователя.
- 4 Пользовательские настройки (громкость, клавиши управления).
- 5 Никнейм игрока для сохранения результата.

1.2.4 Выходные данные.

- 1 Визуальное отображение игровых объектов.
- 2 Информация о точности нажатий.
- 3 Отображение текущего количества очков и НР.
- 4 Сообщения о начале и завершении игры.
- 5 Итоговый результат прохождения композиции.
- 6 Таблица лидеров, полученная с удалённого сервера.

1.2.5 Требования к временным характеристикам.

- 1 Запуск воспроизведения музыки — не более 1 секунды.
- 2 Обработка пользовательского ввода — в режиме реального времени.
- 3 Обновление визуальных элементов — не реже 60 кадров в секунду.

4 Подсчет результатов нажатия — не более 10 миллисекунд.

5 Получение и отправка данных таблицы лидеров — не более 5 секунд при наличии интернет-соединения.

1.2.6 Требования к надежности.

1 Корректная обработка некорректных или поврежденных файлов.

2 Корректная обработка некорректных или поврежденных файлов.

3 Стабильная работа при повторных запусках игры.

4 Предотвращение зависаний при ошибках ввода.

5 Корректная обработка сетевых ошибок и отсутствия подключения к интернету.

6 Сохранение работоспособности локального игрового процесса при недоступности сервера.

1.2.7 Условия эксплуатации.

Программа предназначена для работы на персональном компьютере с возможностью подключения к сети Интернет для использования сетевой таблицы лидеров.

Поддерживаемые операционные системы:

1 Windows 11 (64-bit).

2 Windows 10 (64-bit).

3 Windows 7 (64-bit).

Управление осуществляется с помощью клавиатуры. Графический интерфейс отображается в оконном режиме и адаптируется под размеры экрана. Для работы сетевого рейтинга требуется стабильное подключение к сети Интернет.

2 АНАЛИЗ ТРЕБОВАНИЙ К ПС И РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

2.1 Описание функциональности ПС

Всевозможные акторы:

2.1.1 Загрузка карты и запуск игры.

Акторы: пользователь, программа.

Цель: загрузка музыкальной композиции и соответствующей карты таймингов, подготовка к началу игры.

Предусловия: программа запущена, игра находится в основном меню.

Основной сценарий:

- 1 Пользователь выбирает пункт меню «Загрузить карту».
- 2 Пользователь указывает файл карты таймингов.
- 3 Программа автоматически определяет и загружает связанный музыкальный файл.
- 4 Программа проверяет корректность загруженных данных.
- 5 После успешной загрузки программа отображает информацию о карте и готовности к игре.

Альтернативные сценарии:

- 1 Файл карты отсутствует или поврежден — программа уведомляет пользователя об ошибке.
- 2 Музыкальный файл не найден — программа сообщает о невозможности запуска игры.

2.1.2 Подготовка и запуск игрового процесса.

Акторы: пользователь, программа.

Цель: подготовка игрока к началу ритмического взаимодействия.

Предусловия: карта и музыкальный файл успешно загружены.

Основной сценарий:

- 1 Пользователь выбирает пункт «Начать игру».
- 2 Программа запускает подготовительный этап.
- 3 На экране отображается игровая зона и элементы интерфейса.
- 4 Через заданную задержку начинается воспроизведение музыки и активируется игровой процесс.

Альтернативные сценарии:

- 1 Пользователь пытается начать игру без загруженной карты — программа выводит предупреждение.

2.1.3 Основной игровой процесс.

Актёры: пользователь, программа.

Цель: выполнение нажатий клавиш в такт музыкальной композиции.

Предусловия: игровой процесс запущен, музыка воспроизводится.

Основной сценарий:

1 Программа отображает игровые объекты, движущиеся в сторону зоны попадания.

2 Пользователь нажимает назначенные клавиши в момент совпадения объекта с зоной попадания.

3 Программа фиксирует время нажатия и сравнивает его с тайм-кодом ноты.

4 В зависимости от точности нажатия определяется результат (отлично, хорошо, допустимо, промах).

5 Программа обновляет счет, здоровье и визуальную обратную связь.

Альтернативные сценарии:

1 Пользователь нажимает клавишу вне допустимого временного интервала — фиксируется промах.

2 Пользователь не нажимает клавишу — нота автоматически засчитывается как промах.

2.1.4 Подсчет результатов и завершение игры.

Актёры: программа.

Цель: определение итогового результата прохождения композиции и сохранение его в сетевой таблице лидеров.

Предусловия: все ноты карты обработаны.

Основной сценарий:

1 Программа анализирует все зарегистрированные нажатия.

2 Итоговый результат отображается на экране.

3 Программа завершает воспроизведение музыки.

4 Пользователю предлагается ввести никнейм для сохранения результата.

5 Программа отправляет результат в удалённую базу данных.

6 После успешной отправки программа обновляет сетевую таблицу лидеров.

Альтернативные сценарии:

1 Значение здоровья игрока достигает нуля — игра завершается досрочно с сообщением о поражении.

2 Пользователь отменяет ввод никнейма — результат не сохраняется.

2.1.5 Загрузка и отображение сетевой таблицы лидеров

Акторы: пользователь, программа, удалённая база данных.

Цель: получение и отображение результатов игроков через сеть Интернет.

Предусловия: программа имеет доступ к сети Интернет.

Основной сценарий:

- 1 Пользователь выбирает пункт меню «Лидеры».
- 2 Программа отправляет HTTP GET-запрос к удалённой базе данных.
- 3 Сервер возвращает данные в формате JSON.
- 4 Программа обрабатывает полученные данные.
- 5 Результаты сортируются по количеству очков.
- 6 Таблица лидеров отображается пользователю.

Альтернативные сценарии:

- 1 Сервер недоступен — программа выводит сообщение об ошибке соединения.
- 2 Таблица лидеров пуста — программа отображает соответствующее уведомление.

2.1.6 Изменение настроек.

Акторы: пользователь, программа.

Цель: настройка параметров управления и звука.

Предусловия: программа запущена.

Основной сценарий:

- 1 Пользователь открывает окно настроек.
- 2 Пользователь изменяет громкость музыки и звуковых эффектов.
- 3 Пользователь назначает клавиши управления.
- 4 Программа сохраняет настройки и применяет их к игровому процессу.

Альтернативные сценарии:

- 1 Пользователь назначает недопустимую клавишу — программа запрашивает повторный ввод.

2.2 Спецификация функциональных требований

2.2.1 Загрузка карты и запуск игры.

1 Пользователь должен иметь возможность загружать карты таймингов из текстовых файлов.

2 Программа должна автоматически загружать соответствующий музыкальный файл.

3 Программа должна проверять корректность входных данных.

4 При ошибке загрузки программа должна уведомлять пользователя.

2.2.2 Подготовка и запуск игрового процесса.

- 1 Программа должна запускать игру только при наличии загруженной карты.
- 2 Должен присутствовать подготовительный интервал перед началом воспроизведения музыки.
- 3 Программа должна отображать текущее состояние игры.

2.2.3 Основной игровой процесс.

- 1 Программа должна отображать игровые объекты, синхронизированные с музыкой.
- 2 Программа должна обрабатывать нажатия клавиш в режиме реального времени.
- 3 Точность нажатий должна оцениваться по заданным временным интервалам.
- 4 Программа должна учитывать промахи при отсутствии нажатия.
- 5 Программа должна воспроизводить звуковую и визуальную обратную связь.
- 6 Программа должна подсчитывать количество очков пользователя.

2.2.4 Подсчет результатов и завершение игры.

- 1 Программа должна вести подсчет очков и уровня здоровья игрока.
- 2 При снижении уровня здоровья до нуля игра должна завершаться досрочно.
- 3 После завершения композиции программа должна отображать итоговый результат.
- 4 Программа должна предоставлять возможность сохранения результата в сетевой таблице лидеров.

2.2.5 Сетевая таблица лидеров.

- 1 Программа должна поддерживать отправку результатов игроков на удалённый сервер.
- 2 Для обмена данными должны использоваться HTTP-запросы.
- 3 Данные должны передаваться в формате JSON.
- 4 Программа должна загружать таблицу лидеров из удалённой базы данных.
- 5 Результаты игроков должны сортироваться по количеству очков.
- 6 При совпадении никнейма программа должна обновлять результат только при получении большего количества очков.
- 7 При сетевых ошибках программа должна уведомлять пользователя.

2.2.6 Настройки.

- 1 Программа должна предоставлять интерфейс для изменения громкости музыки.
- 2 Программа должна предоставлять возможность изменения громкости звуковых эффектов.
- 3 Программа должна позволять переназначать клавиши управления.
- 4 Настройки должны сохраняться между запусками программы.

3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

3.1 Разработка алгоритма ПС

Алгоритм работы программы можно представить в виде последовательности действий:

- 1 Запуск программы и инициализация основных компонентов.
- 2 Загрузка карты таймингов и музыкальной композиции.
- 3 Подготовка игрового процесса.
- 4 Основной игровой цикл.
- 5 Обработка нажатий клавиш и оценка точности.
- 6 Подсчет результатов и контроль состояния игрока.
- 7 Отправка результата в сетевую таблицу лидеров.
- 8 Получение и отображение таблицы лидеров.
- 9 Завершение игры и отображение итогов.

3.2 Алгоритмы отдельных модулей

3.2.1 Алгоритм загрузки карты и подготовки игры

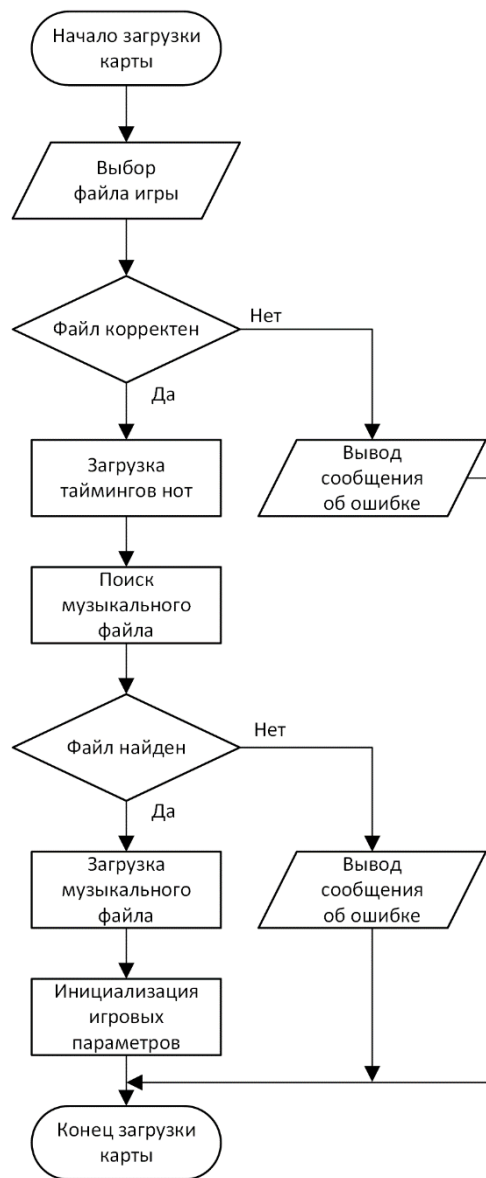


Рисунок 3.1 – Загрузка карты и подготовка игры

3.2.2 Алгоритм запуска и синхронизации игрового процесса

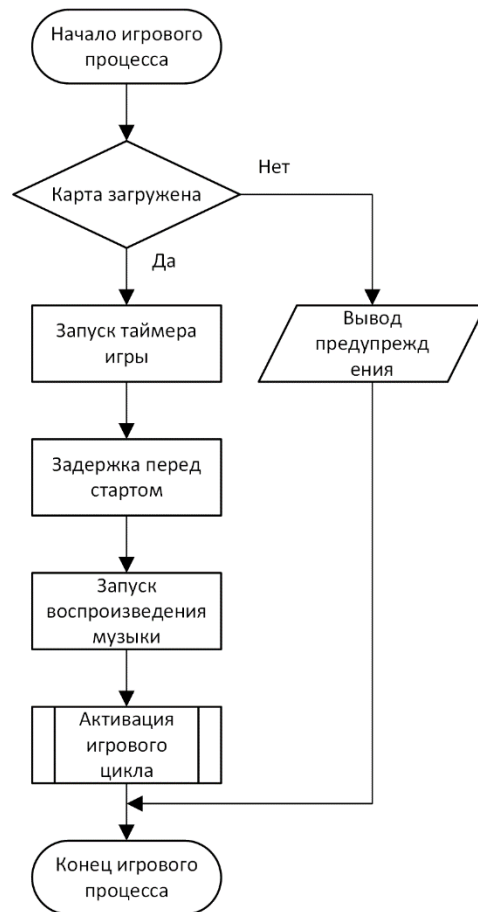


Рисунок 3.2 – Запуск и синхронизация игры

3.2.3 Алгоритм основного игрового цикла

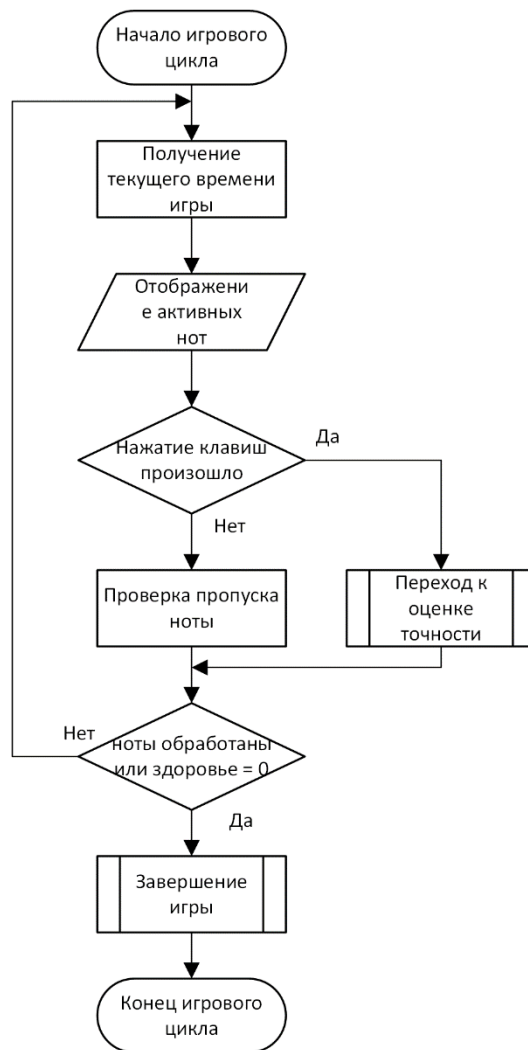


Рисунок 3.3 – Основной игровой цикл

3.2.4 Алгоритм оценки точности

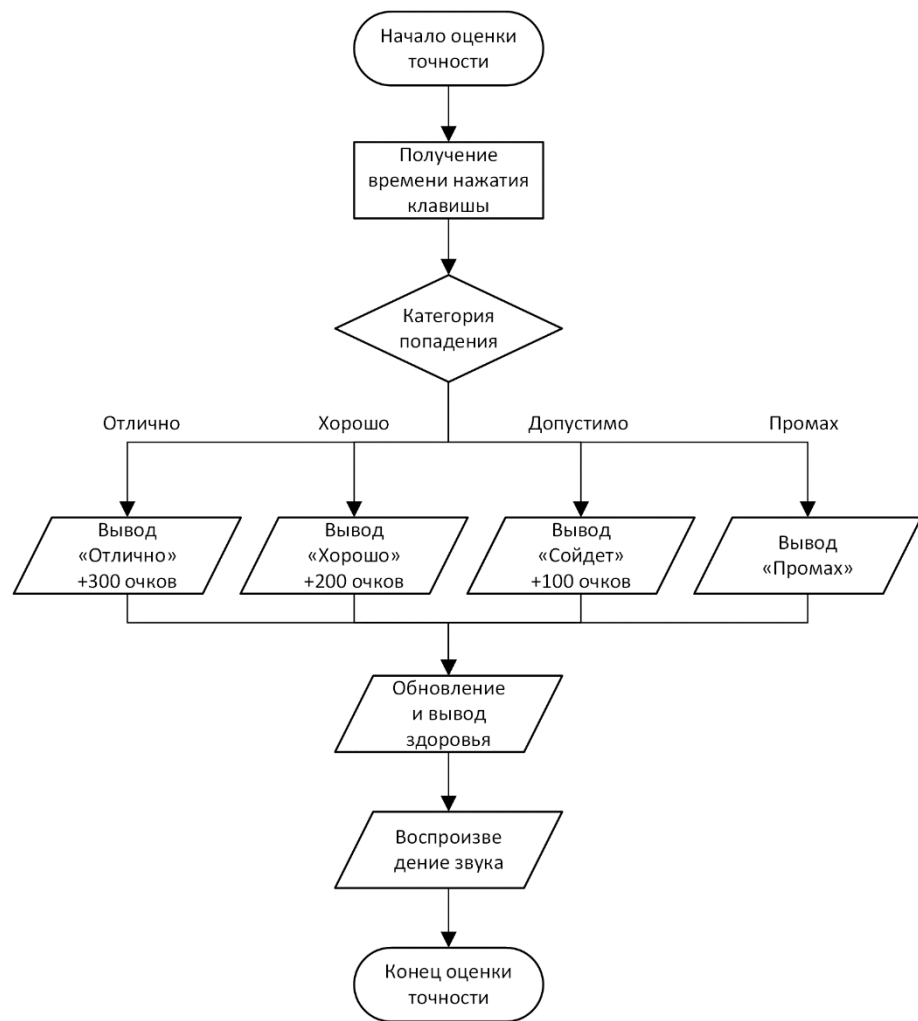


Рисунок 3.4 – Оценка попаданий

3.2.5 Алгоритм корректного завершения игры

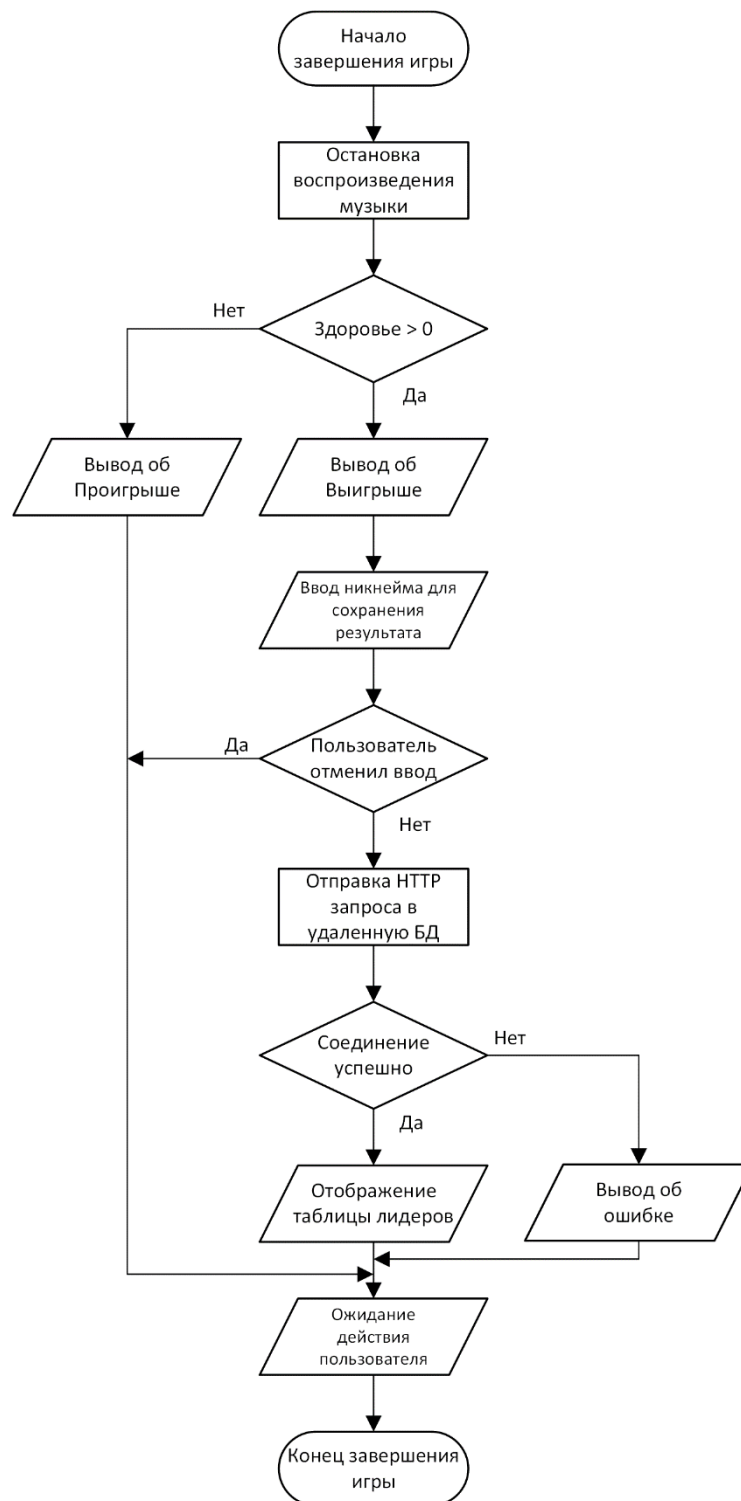


Рисунок 3.5 – Завершение игры

4 СОЗДАНИЕ (КОНСТРУИРОВАНИЕ) ПРОГРАММНОГО СРЕДСТВА

4.1 Структура программного средства

На основании разработанной архитектуры и алгоритмов реализуется программное средство «BeatKey». В процессе реализации осуществляется разработка всех основных модулей, их отладка и интеграция в единое приложение. Программное средство создаётся с использованием модульного подхода, что обеспечивает простоту сопровождения, возможность расширения функционала и повторного использования компонентов в будущем.

Архитектура приложения построена на модульной основе и включает следующие ключевые модули:

1 Модуль основного окна (MainWindow) – отвечает за управление интерфейсом пользователя, отображение игровых объектов, а также обработку событий клавиатуры. Этот модуль контролирует игровой цикл, синхронизацию нот с музыкальным сопровождением и визуализацию анимаций, таких как эффекты попаданий и промахов.

2 Модуль конфигурации игры (GameConfig) – обеспечивает хранение и загрузку настроек игры, включая громкость звуков (hit, miss, музыка) и пользовательские клавиши управления. Модуль реализован с использованием паттерна «синглтон», что гарантирует доступ к конфигурации из любого места программы и упрощает сохранение и изменение настроек.

3 Модуль диалога настроек (SettingsDialog) – предоставляет пользователю интерфейс для изменения параметров игры. Он включает слайдеры для регулировки громкости звуков и музыки, кнопки для изменения основных и дополнительных клавиш управления. Модуль также обрабатывает события ввода и обеспечивает обратную связь в реальном времени, обновляя текущие настройки при изменении пользователем.

4 Модуль работы с музыкой и звуковыми эффектами – реализует воспроизведение музыкальных треков и звуков hit/miss с корректной синхронизацией с игровым циклом. Модуль использует возможности фреймворка Qt (QMediaPlayer, QAudioOutput, QSoundEffect) для точного управления аудио и поддержки различных форматов файлов.

5 Модуль карты ритма (BeatMap) – отвечает за хранение тайм-кодов нот, их состояния (попадание, промах) и обработку игрового времени. Модуль обеспечивает возможность загрузки пользовательских карт из текстовых файлов и корректную синхронизацию нот с музыкальной дорожкой.

6 Модуль сетевого взаимодействия (LeaderboardClient) – реализует взаимодействие приложения с удалённой базой данных Firebase Realtime Database. Модуль обеспечивает отправку и получение данных посредством HTTP-запросов, а также обработку ответов сервера в формате JSON. Для реализации сетевого взаимодействия используется класс QNetworkAccessManager из модуля Qt Network.

4.2 Описание логики, процедур и функций

4.2.1 Основные типы и структуры данных.

В модуле MainWindow определён основной тип данных Note, обозначающий игровую ноту. Он содержит следующие поля:

1 beatTime – момент времени (в миллисекундах), когда нота должна быть нажата.

2 hit – логическое значение, показывающее, была ли нота успешно сыграна.

3 missed – логическое значение, показывающее, была ли нота промахнута.

Также в модуле MainWindow определена структура BeatMap, содержащая:

1 musicPath – путь к музыкальному файлу карты;

2 id – идентификатор карты, используемый для сетевой таблицы лидеров.

3 notes – массив объектов Note, формирующий последовательность ритма.

В модуле GameConfig используется структура хранения настроек игры, включающая:

1 hitVolume, missVolume, musicVolume – значения громкости звуков hit, miss и фоновой музыки;

2 primaryKey, secondaryKey – коды клавиш управления;

3 m_settings – объект QSettings для сохранения и загрузки конфигурации из файла settings.ini.

В модуле SettingsDialog используются переменные primaryKeyCode и secondaryKeyCode для хранения временных значений клавиш, назначаемых пользователем через интерфейс диалога настроек.

В модуле LeaderboardClient используется структура LeaderboardEntry, содержащая информацию о записи таблицы лидеров:

1 nickname – имя игрока.

2 score – количество набранных очков.

3 updatedAt – дата и время обновления результата.

4.2.2 Подпрограммы модуля MainWindow.

Таблица 4.1 – Описание процедур и функций модуля MainWindow

Имя подпрограммы	Описание	Заголовок	Имя параметра	Назначение параметра
loadMap()	Загружает карту из текстового файла, создаёт объекты Note и синхронизирует с музыкальным файлом	void MainWindow::loadMap()	—	—
startGame()	Запускает игровой цикл, сбрасывает состояние нот и HP	void MainWindow::startGame()	—	—
updateGame()	Обновляет состояние игры каждое тактовое событие таймера, запускает музыку, проверяет попадания и промахи	void MainWindow::updateGame()	—	—
keyPressEvent(QKeyEvent *event)	Обрабатывает нажатия клавиш, проверяет попадания в ноты, регистрирует результат	void MainWindow::keyPressEvent(QKeyEvent *event)	event	Содержит информацию о нажатой клавише
registerHit(qint64 diff)	Рассчитывает результат попадания (отлично, хорошо, сойдёт, промах), обновляет HP и отображает анимацию	void MainWindow::registerHit(qint64 diff)	diff	Разница между временем ноты и временем нажатия клавиши
animateScoreLabel()	Запускает анимацию изменения положения текста оценки	void MainWindow::animateScoreLabel()	—	—
updateHpLabel()	Обновляет текст и цвет HP	void MainWindow::updateHpLabel()	—	—

Продолжение таблицы 4.1

updateScoreStatusLabel()	Обновляет отображение количества очков	количества очков void MainWindow::updateScoreStatusLabel()	—	—
submitScore()	Запрашивает никнейм игрока и отправляет результат в таблицу лидеров	void MainWindow::submitScore()	—	—
showLeaderboard()	Отображает окно сетевой таблицы лидеров	void MainWindow::showLeaderboard()	—	—
onLeaderboardFetched(...)	Обрабатывает полученные данные таблицы лидеров	void MainWindow::onLeaderboardFetched(...)	entries	Список записей таблицы лидеров
onLeaderboardSubmitFinished(...)	Обрабатывает результат отправки очков на сервер	void MainWindow::onLeaderboardSubmitFinished(...)	success, message	Результат операции и текст уведомления
showSettingsDialog()	Отображает диалог настроек и передаёт текущие значения	void MainWindow::showSettingsDialog()	—	—
applySettings()	Применяет изменения из диалога настроек к игре	void MainWindow::applySettings()	—	—

4.2.3 Подпрограммы модуля SettingsDialog.

Таблица 4.2 – Описание процедур и функций модуля SettingsDialog

Имя подпрограммы	Описание	Заголовок	Имя параметра	Назначение параметра
hitVolume()	Возвращает текущую громкость звука hit	float SettingsDialog::hitVolume()	—	—

Продолжение таблицы 4.2

missVolume()	Возвращает текущую громкость звука miss	float SettingsDialog::missVolume()	—	—
musicVolume()	Возвращает текущую громкость музыки	float SettingsDialog::musicVolume()	—	—
primaryKey()	Возвращает код основной клавиши	int SettingsDialog::primaryKey()	—	—
secondaryKey()	Возвращает код дополнительной клавиши	int SettingsDialog::secondaryKey()	—	—
setHitVolume(float volume)	Устанавливает громкость звука hit	void SettingsDialog::setHitVolume(float volume)	volume	Значение громкости от 0 до 1
setMissVolume(float volume)	Устанавливает громкость звука miss	void SettingsDialog::setMissVolume(float volume)	volume	Значение громкости от 0 до 1
setMusicVolume(float volume)	Устанавливает громкость музыки	void SettingsDialog::setMusicVolume(float volume)	volume	Значение громкости от 0 до 1
setPrimaryKey(int key)	Устанавливает основную клавишу управления	void SettingsDialog::setPrimaryKey(int key)	key	Код клавиши
setSecondaryKey(int key)	Устанавливает дополнительную клавишу управления	void SettingsDialog::setSecondaryKey(int key)	key	Код клавиши
keyPressEvent(QKeyEvent *event)	Обработывает нажатия клавиш для назначения биндов	void SettingsDialog::keyPressEvent(QKeyEvent *event)	event	Содержит информацию о нажатой клавише

Продолжение таблицы 4.2

onPrimary KeyButton Clicked()	Переводит кнопку назначения основной клавиши в режим ожидания	void SettingsDialog::onPri maryKeyButtonClick ed()	—	—
onSecondar yKeyButto nClicked()	Переводит кнопку назначения дополнительной клавиши в режим ожидания	void SettingsDialog::onSe condaryKeyButtonCli cked()	—	—
updateKey ButtonText (QPushButton *button, int key)	Обновляет текст кнопки, отображающий текущую клавишу	void SettingsDialog::updat eKeyButtonText(QPu shButton *button, int key)	button , key	Кнопка интерфейса и код клавиши

4.2.4 Подпрограммы модуля SettingsDialog.

Таблица 4.3 – Описание процедур и функций модуля LeaderboardClient

Имя подпрог раммы	Описание	Заголовок	Имя парамет ра	Назначение параметра
fetch(QS tring mapId)	Загружает таблицу лидеров выбранной карты	void LeaderboardCli ent::fetch(QStri ng mapId)	mapId	Идентификато р карты
submit(QString mapId, QString nicknam e, int score)	Отправляет результат игрока в удалённую базу данных	void LeaderboardCli ent::submit(QS tring mapId, QString nickname, int score)	mapId, nicknam e, score	Идентификато р карты, никнейм игрока и количество очков
mapUrl(QString mapId)	Формирует URL таблицы лидеров карты	QString LeaderboardCli ent::mapUrl(Q String mapId)	mapId	Идентификато р карты

Продолжение таблицы 4.3

playerUrl(QString mapId, QString nickname)	Формирует URL записи конкретного игрока	QString LeaderboardClient::playerUrl(QString mapId, QString nickname)	mapId, nickname	Идентификатор карты и никнейм игрока
parseLeaderboard(QJsonDocument doc)	Преобразует JSON-документ в список результатов	QVector LeaderboardClient::parseLeaderboard(QJsonDocument doc)	doc	JSON-документ с результатами
sortLeaderboard(..)	Сортирует результаты игроков по количеству очков	void LeaderboardClient::sortLeaderboard(...)	entries	Массив записей таблицы лидеров

5 ТЕСТИРОВАНИЕ, ПРОВЕРКА РАБОТОСПОСОБНОСТИ И АНАЛИЗ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ

5.1 Тестирование программного средства

Тестирование проводилось вручную и частично автоматизировано в среде исполнения приложения. Были сформированы сценарии, имитирующие действия пользователей в различных условиях, включая стандартные и экстремальные ситуации. Ниже приведены примеры тестов, проведённых для проверки основных функций BeatKey.

Таблица 5.1 – Примеры проведённых тестов

№	Описание теста	Входные данные	Ожидаемый результат	Фактический результат
1	Загрузка корректной карты	Текстовый файл с нотами и mp3 рядом	Карта успешно загружается	Успешно
2	Загрузка карты без музыкального файла	txt-файл без mp3	Сообщение об ошибке загрузки	Успешно
3	Попытка запуска игры без карты	–	Игра не запускается	Успешно
4	Попадание в интервал ± 25 мс	Нажатие в точный тайминг	Отображается «ОТЛИЧНО», начисляется 300 очков	Успешно
5	Попадание в интервал ± 50 мс	Нажатие с небольшой погрешностью	Отображается «ХОРОШО», начисляется 200 очков	Успешно
6	Попадание в интервал ± 100 мс	Позднее или раннее нажатие	Отображается «СОЙДЁТ», начисляется 100 очков	Успешно
7	Пропуск ноты	Отсутствие нажатия	Засчитывается «ПРОМАХ», НР уменьшается	Успешно
8	Нажатие вне игрового интервала	Произвольное нажатие	Попадание не засчитывается	Успешно
9	Проверка окончания игры при НР = 0	Серия промахов	Игра завершается сообщением о поражении	Успешно

Продолжение таблицы 5.1

10	Проверка сохранения настроек	Изменение клавиш и громкости	Настройки сохраняются после перезапуска	Успешно
11	Изменение клавиш управления	Назначение клавиш Z и X	Игра корректно реагирует на новые клавиши	Успешно
12	Изменение громкости музыки	musicVolume = 0.2	Громкость музыки уменьшается	Успешно
13	Проверка анимации scoreLabel	Успешное попадание	Текст анимируется корректно	Успешно
14	Проверка адаптации интерфейса	Изменение размера окна	Элементы интерфейса корректно repositionируются	Успешно
15	Работа нескольких нот подряд	Быстрая последовательность нот	Все ноты корректно обрабатываются	Успешно
16	Проверка запуска музыки	Запуск игры	Музыка начинается после preStartDelay	Успешно
17	Проверка загрузки таблицы лидеров	Нажатие кнопки «Лидеры»	Отображается список рекордов	Успешно
18	Отправка нового результата	Новый никнейм и результат	Результат сохраняется в Firebase	Успешно
19	Обновление существующего рекорда	Результат выше предыдущего	Старый рекорд заменяется новым	Успешно
20	Отправка худшего результата	Результат ниже существующего	Выводится предупреждение, запись не изменяется	Успешно

5.2 Экспериментальная проверка

Для проверки корректности игрового процесса и сетевого взаимодействия программа была протестирована в условиях реального использования. В тестировании принимали участие пользователи различного уровня подготовки. Проверялись как игровые механики, так и работа таблицы лидеров при наличии подключения к сети Интернет. Результаты показали:

1 Игра стабильно воспроизводит музыкальные композиции и синхронизирует ноты с аудиопотоком.

2 Все этапы игрового процесса понятны пользователю и не вызывают затруднений.

3 Система оценки точности корректно реагирует на различные интервалы нажатий.

4 Подсчёт очков и изменение HP выполняются без ошибок.

5 Сетевое взаимодействие с Firebase Realtime Database выполняется корректно.

6 Таблица лидеров успешно загружается и обновляется через HTTP-запросы.

7 Обновление существующего рекорда работает только при превышении предыдущего результата.

8 Интерфейс программы остаётся отзывчивым даже при выполнении сетевых операций.

5.3 Анализ полученных данных

В ходе тестирования были проанализированы основные механизмы приложения и стабильность его работы. Полученные результаты позволяют сделать следующие выводы:

1 Синхронизация игровых объектов с музыкальной композицией реализована корректно, а задержка обработки ввода не оказывает заметного влияния на игровой процесс.

2 Система оценки попаданий корректно распределяет результаты по временным интервалам и правильно начисляет очки.

3 Система сохранения пользовательских настроек обеспечивает стабильное хранение параметров между запусками программы.

4 Модуль сетевого взаимодействия успешно выполняет HTTP GET и PUT запросы к Firebase Realtime Database.

5 JSON-структуры корректно сериализуются и десериализуются средствами Qt.

6 Таблица лидеров корректно сортируется по количеству очков и отображает актуальные результаты игроков.

7 Программа устойчива к некорректному вводу, отсутствующим файлам и ошибкам сетевого взаимодействия.

5.4 Результаты тестирования

Проведённое тестирование подтвердило соответствие программного средства BeatKey требованиям технического задания. Все основные функции приложения реализованы и работают корректно. Игровой процесс сопровождается стабильной обработкой ввода, корректной синхронизацией музыки и нот, а также адекватной системой оценки попаданий.

Сетевая подсистема, реализованная с использованием Qt Network и Firebase Realtime Database, обеспечивает корректную загрузку и обновление таблицы лидеров. Обмен данными выполняется в формате JSON посредством HTTP-запросов, что подтверждает работоспособность сетевой части программного средства.

Интерфейс приложения остаётся стабильным и отзывчивым на всех этапах работы программы. Выявленные в ходе тестирования незначительные недочёты пользовательского интерфейса были устранены в процессе отладки.

Таким образом, можно сделать вывод, что программное средство BeatKey успешно прошло проверку работоспособности и готово к использованию в учебных, демонстрационных и развлекательных целях.

6 РУКОВОДСТВО ПО УСТАНОВКЕ И ИСПОЛЬЗОВАНИЮ

6.1 Установка приложения “BeatKey”

Для загрузки программного обеспечения необходимо перейти по предоставленной ссылке[5]. Она направляет на официальную страницу проекта на платформе GitHub.

На данной странице требуется найти и скачать архив программы в формате ZIP:

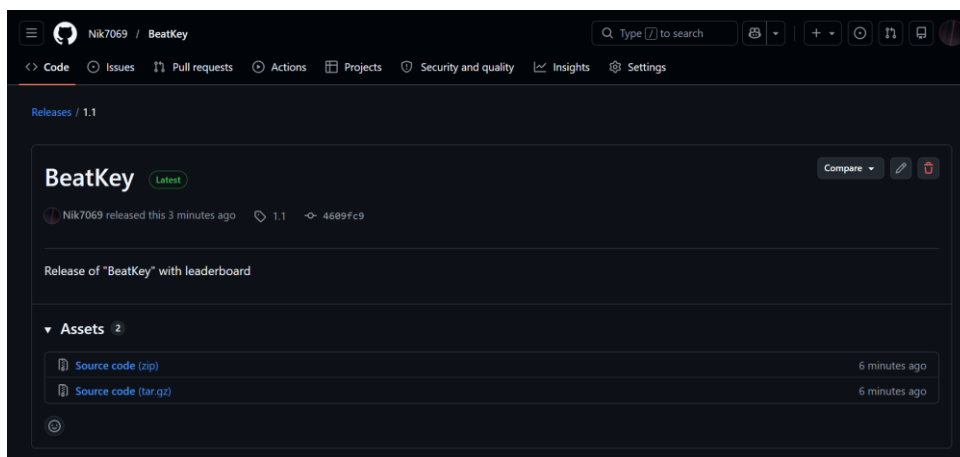


Рисунок 6.1 – Страница для скачивания приложения

После завершения загрузки ZIP-файла его необходимо распаковать в выбранную директорию. В результате разархивации должна быть создана папка, содержащая следующие файлы:

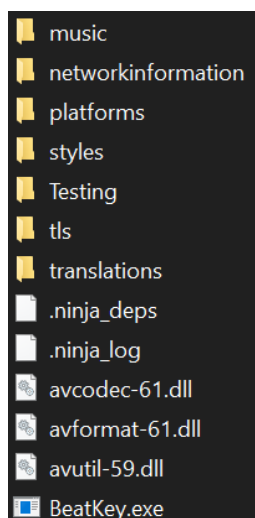


Рисунок 6.2 – Примерные файлы программы

Для начала работы с приложением следует открыть распакованную папку и запустить исполняемый файл "BeatKey.exe". После успешного запуска программа будет готова к эксплуатации. Для корректной работы сетевой таблицы лидеров требуется наличие подключения к сети Интернет. При отсутствии подключения игровой процесс остаётся полностью работоспособным, однако функции загрузки и отправки результатов становятся недоступны.

6.2 Эксплуатация приложения "BeatKey"

Программа BeatKey рассчитана на использование на одном устройстве (например, ноутбуке, настольном ПК или интерактивной панели) и подходит для игры одного пользователя или нескольких участников по очереди.

В главном меню располагаются кнопки «Загрузить карту», «Начать игру». При запуске игры пользователь:

- 1 Загружает предварительно подготовленную карту с нотами и музыкальным треком.

- 2 Выбирает или оставляет стандартные клавиши управления (primaryKey и secondaryKey).

- 3 Следует правилам: вовремя нажимать клавиши, соответствующие нотам, и избегать промахов, чтобы сохранять НР.

Цель игры – набрать максимальное количество очков, точно попадая в ноты, не теряя НР. Игровой процесс условно делится на несколько этапов.

В этапе подготовки и загрузки карты:

- 1 Игрок выбирает файл карты (.txt) с нотами.

- 2 Программа автоматически ищет музыкальный трек (.mp3) с таким же именем в той же папке.

- 3 Пользователь может изменить настройки громкости и клавиш через меню «Звуки и бинды».

- 4 После загрузки карты отображается статус «Карта загружена», можно переходить к следующему этапу.

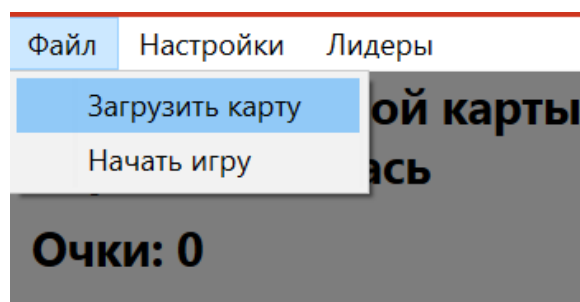


Рисунок 6.3 – Окно загрузки карты

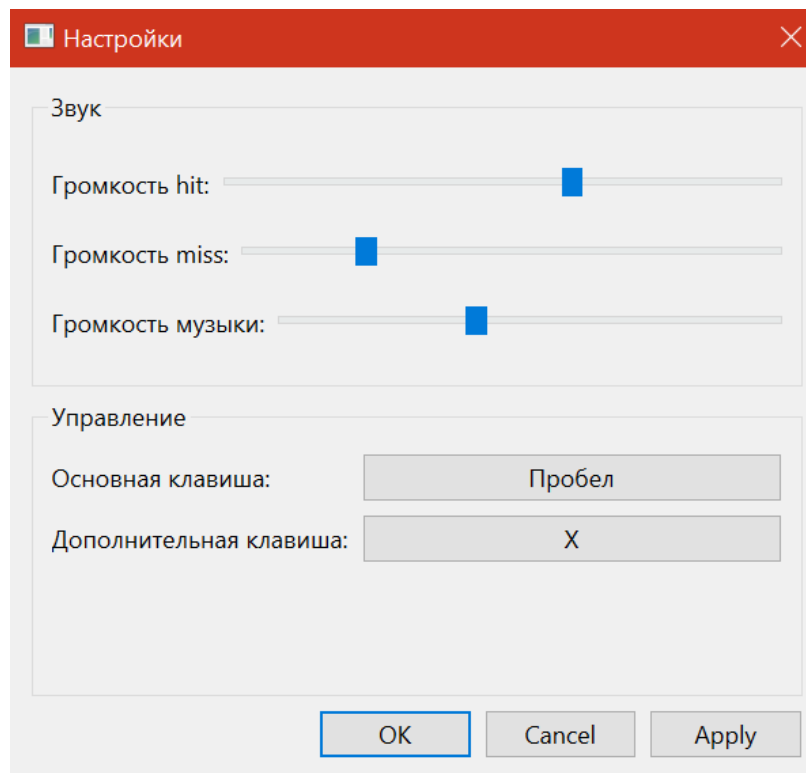


Рисунок 6.4 – Окно настроек звука и управления

На этапе игрового процесса:

- 1 Игра начинает отсчет времени, давая игроку возможность подготовиться.
- 2 Ноты начинают двигаться к hit-зоне справа налево.
- 3 Игрок должен нажимать клавиши `primaryKey` или `secondaryKey`, когда нота находится в зоне попадания.
- 4 Программа отображает текстовые оценки попаданий: «ОТЛИЧНО», «ХОРОШО», «СОЙДЕТ» или «ПРОМАХ» и дает очки согласно оценке попадания (300, 200, 100 и 0 соответственно).
- 5 HP изменяется в зависимости от точности попаданий; если HP достигает 0, игра завершается.

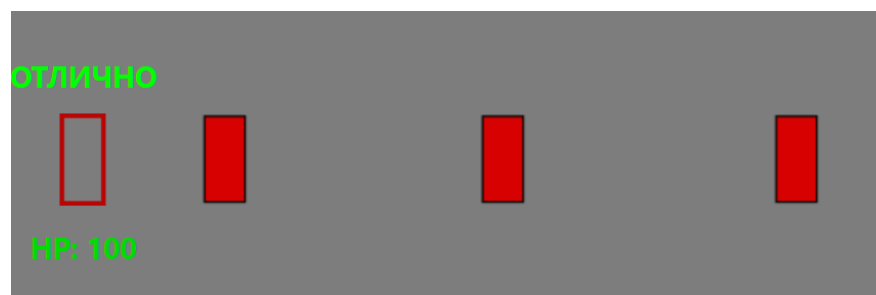


Рисунок 6.5 – Основное игровое окно

В этапе завершения игры:

- 1 Игра автоматически заканчивается после прохождения всех нот.
- 2 Программа выводит финальные результаты: последняя точность попаданий и оставшийся HP.
- 3 Пользователю предлагается ввести никнейм для отправки результата в сетевую таблицу лидеров.
- 4 При наличии подключения к Интернету результат отправляется в Firebase Realtime Database.
- 5 Пользователь может просмотреть таблицу лидеров через пункт меню «Лидеры».



Рисунок 6.6 – Завершение игры и отображение результатов

Лидеры: bleenSong		
Место	Никнейм	Очки
1	ProGamer	19400
2	/%XxLuc@sxX%\	19200
3	Hacker	18400
4	Test	15500
5	MiddlePlayerYeeeah	14500
6	Test2	10100
7	Loser	0
OK		

Рисунок 6.7 – Таблица лидеров песни “bleenSong”

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано игровое программное средство BeatKey, реализующее механику ритмического взаимодействия пользователя с музыкальными композициями и поддерживающее сетевую систему рейтинга игроков.

Программное средство реализовано на языке программирования C++ с использованием фреймворка Qt. В процессе разработки были использованы компоненты Qt Widgets, Qt Multimedia и Qt Network, что позволило реализовать графический интерфейс, воспроизведение музыки, обработку пользовательского ввода и сетевое взаимодействие с удалённой базой данных.

В процессе работы были решены следующие задачи:

1 Проведён анализ существующих ритм-игр и сформированы требования к разрабатываемому программному средству.

2 Спроектирована архитектура приложения и алгоритмы обработки игрового процесса.

3 Реализована система синхронизации нот с музыкальным сопровождением.

4 Разработана система оценки точности попаданий, подсчёта очков и управления уровнем здоровья игрока.

5 Реализован модуль пользовательских настроек с поддержкой сохранения параметров между запусками программы.

6 Разработана сетевая таблица лидеров с использованием Firebase Realtime Database.

7 Реализован обмен данными посредством HTTP GET и PUT запросов в формате JSON.

8 Проведено тестирование приложения в различных сценариях эксплуатации.

Разработанное программное средство обеспечивает стабильную работу игрового процесса, корректную обработку пользовательского ввода и синхронизацию игровых объектов с музыкальным сопровождением. Сетевая подсистема позволяет сохранять и загружать результаты игроков через Интернет, обеспечивая возможность формирования общего рейтинга пользователей.

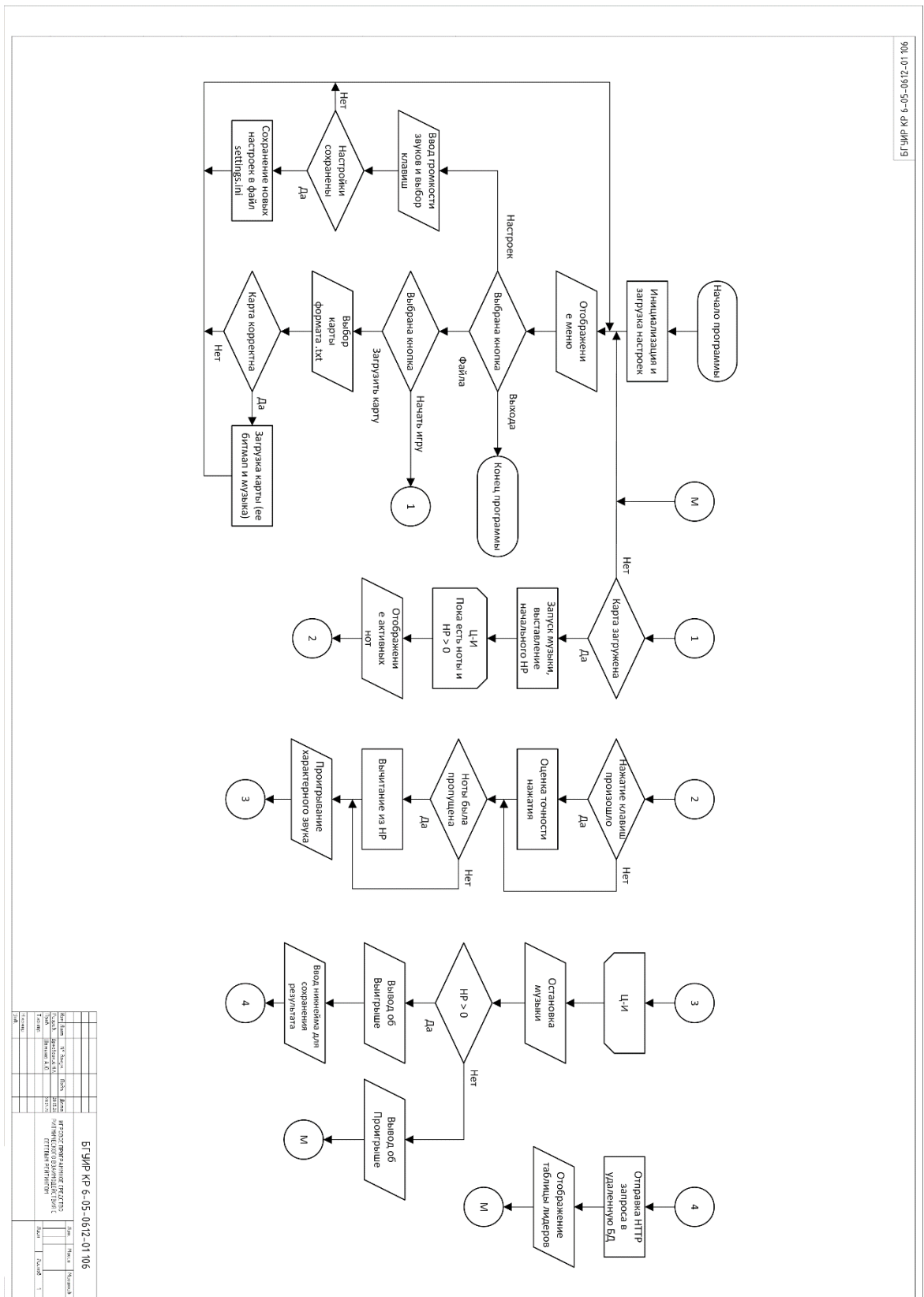
Таким образом, поставленные цели курсовой работы были полностью достигнуты. Результатом работы является полностью функционирующее игровое приложение BeatKey, пригодное для использования в учебных, демонстрационных и развлекательных целях.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] osu! [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://osu.ppy.sh>.
- [2] Friday Night Funkin' [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://ninja-muffin24.itch.io/funkin>.
- [3] A Dance of Fire and Ice [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://fizzd.itch.io/a-dance-of-fire-and-ice>.
- [4] Firebase Realtime Database [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://firebase.google.com>
- [5] Страница проекта “BeatKey” на платформе GitHub [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://github.com/Nik7069/BeatKey/releases/latest>.

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А



ПРИЛОЖЕНИЕ Б

main.cpp

```
#include "mainwindow.h"

#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;
    w.show();
    return a.exec();
}
```

mainwindow.cpp

```
#include "mainwindow.h"
#include "../ui_mainwindow.h"

#include <QFileDialog>
#include <QKeyEvent>
#include <QDebug>
#include <QPainter>
#include <QPen>
#include <QInputDialog>
#include <QMessageBox>
#include <QTableWidget>
#include <QHeaderView>
#include <QVBoxLayout>
#include <QDialogButtonBox>
#include <QDialog>
#include <QLineEdit>

MainWindow::MainWindow(QWidget *parent)
: QMainWindow(parent)
, ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    scoreStatusLabel = new QLabel(ui->centralwidget);
    scoreStatusLabel->setGeometry(10, 56, 400, 30);
    scoreStatusLabel->setFont(ui->statusGameLabel->font());
    scoreStatusLabel->setStyleSheet("color: black; font-weight: bold;");
    scoreStatusLabel->show();
    updateScoreStatusLabel();

    // Загружаем настройки
    config.loadSettings();
    primaryKey = config.primaryKey();
    secondaryKey = config.secondaryKey();

    player = new QMediaPlayer(this);
    audioOutput = new QAudioOutput(this);

    // Устанавливаем громкость из настроек
    audioOutput->setVolume(config.musicVolume());
    player->setAudioOutput(audioOutput);

    // Подключаем действия меню
    connect(ui->actionLoadMap, &QAction::triggered,
            this, &MainWindow::loadMap);
}
```

```

connect(ui->actionPlayGame, &QAction::triggered,
        this, &MainWindow::startGame);

// Добавляем новое действие для настроек (нужно добавить в UI)
// В дизайнера добавьте QAction в меню "Файл" с именем actionSettings
connect(ui->actionSettings, &QAction::triggered,
        this, &MainWindow::showSettingsDialog);

ui->menubar->addAction("Лидеры", this, &MainWindow::showLeaderboard);

leaderboardClient = new LeaderboardClient(this);
connect(leaderboardClient, &LeaderboardClient::fetchFinished,
        this, &MainWindow::showLeaderboardEntries);
connect(leaderboardClient, &LeaderboardClient::submitFinished,
        this, &MainWindow::onLeaderboardSubmitFinished);
connect(leaderboardClient, &LeaderboardClient::requestFailed,
        this, &MainWindow::onLeaderboardRequestFailed);

gameLoopTimer = new QTimer(this);
connect(gameLoopTimer, &QTimer::timeout, this, &MainWindow::updateGame);
gameLoopTimer->start(16);

QPixmap rawNote("assets/note.png");
QPixmap rawHitZone("assets/hitzone.png");

notePixmap = rawNote.scaled(
    hitZoneSize, hitZoneSize,
    Qt::KeepAspectRatio,
    Qt::SmoothTransformation
);

hitZonePixmap = rawHitZone.scaled(
    squareSize, squareSize,
    Qt::KeepAspectRatio,
    Qt::SmoothTransformation
);

if (notePixmap.isNull())
    qDebug() << "Failed to load note.png";
if (hitZonePixmap.isNull())
    qDebug() << "Failed to load hitzone.png";

hitSound.setSource(QUrl::fromLocalFile("assets/hit.wav"));
hitSound.setVolume(config.hitVolume());

missSound.setSource(QUrl::fromLocalFile("assets/miss.wav"));
missSound.setVolume(config.missVolume());

hitSound.setLoopCount(1);
missSound.setLoopCount(1);

scoreAnim = new QPropertyAnimation(ui->scoreLabel, "pos", this);
scoreAnim->setDuration(150);
scoreAnim->setEasingCurve(QEasingCurve::OutCubic);

scoreLabelBasePos = ui->scoreLabel->pos();
}

MainWindow::~MainWindow()
{
    config.saveSettings();
    delete ui;
}

```

```

void MainWindow::keyPressEvent(QKeyEvent *event)
{
    if (gameState != GameState::Playing)
        return;

    // Используем сохраненные клавиши
    int key = event->key();
    if ((key != primaryKey && key != secondaryKey) || event->isAutoRepeat())
        return;

    qint64 gameTime = gameTimer.elapsed();
    qint64 musicTime = gameTime - preStartDelay;

    Note *bestNote = nullptr;
    qint64 bestDiff = LLONG_MAX;
    qint64 diff;

    for (Note &note : currentMap.notes) {
        if (note.hit || note.missed)
            continue;

        diff = musicTime - note.beatTime;

        if (qAbs(diff) < qAbs(bestDiff)) {
            bestDiff = diff;
            bestNote = &note;
        }
    }

    qDebug() << musicTime;

    if (!bestNote)
        return;

    registerHit(bestDiff);

    if (qAbs(bestDiff) <= missAfterTime) {
        bestNote->hit = true;
    }
}

```

```

void MainWindow::loadMap()
{
    QString txtPath = QFileDialog::getOpenFileName(
        this,
        "Load beatmap",
        "",
        "Beatmap (*.txt)"
    );

    if (txtPath.isEmpty())
        return;

    QFile file(txtPath);
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        ui->statusMapLabel->setText("Ошибка загрузки карты");
        return;
    }
}

```

```

currentMap.notes.clear();

QTextStream in(&file);
while (!in.atEnd()) {
    QString line = in.readLine();
    bool ok;
    qint64 t = line.toLongLong(&ok);
    if (ok)
        currentMap.notes.append(Note{ t, false, false });
}

file.close();

// Ищем mp3 рядом
QFileInfo info(txtPath);
QString mp3Path = info.absolutePath() + "/" +
    info.completeBaseName() + ".mp3";

if (!QFile::exists(mp3Path)) {
    ui->statusMapLabel->setText("MP3 не найден рядом с картой");
    return;
}

currentMap.musicPath = mp3Path;
currentMap.id = info.completeBaseName();
currentMap.title = info.completeBaseName();
player->setSource(QUrl::fromLocalFile(mp3Path));

ui->statusMapLabel->setText("Карта загружена: " + info.completeBaseName());
score = 0;
updateScoreStatusLabel();
}

void MainWindow::startGame()
{
    if (currentMap.notes.isEmpty()) {
        ui->statusGameLabel->setText("Карта не загружена");
        return;
    }

    // СБРОС СОСТОЯНИЯ НОТ
    for (Note &note : currentMap.notes) {
        note.hit = false;
        note.missed = false;
    }

    hp = 50;
    score = 0;
    scoreDialogShown = false;
    updateHpLabel();
    updateScoreStatusLabel();
    ui->scoreLabel->setText(QString("Score: %1").arg(score));

    gameState = GameState::Playing;
    ui->statusGameLabel->setText("Подготовка...");

    player->stop();

```



```

    player->setPosition(0);

    gameTimer.restart();
    setFocus();
}

void MainWindow::updateGame()
{
    if (gameState != GameState::Playing)
        return;

    qint64 gameTime = gameTimer.elapsed();
    qint64 musicTime = gameTime - preStartDelay;

    // запуск музыки после подготовки
    if (gameTime >= preStartDelay &&
        player->playbackState() != QMediaPlayer::PlayingState) {

        player->play();
        ui->statusGameLabel->setText("Игра началась");
    }

    // авто-промах для непрожатых нот
    for (Note &note : currentMap.notes) {

        if (note.hit || note.missed)
            continue;

        if (musicTime > note.beatTime + 200) {
            note.missed = true;
            registerHit(9999); // промах
        }
    }

    // конец игры (все ноты обработаны)
    bool allProcessed = true;
    for (const Note &note : currentMap.notes) {
        if (!note.hit && !note.missed) {
            allProcessed = false;
            break;
        }
    }

    if (allProcessed && musicTime > 0) {
        gameState = GameState::Idle;
        player->stop();
        ui->statusGameLabel->setText(QString("Игра окончена. Очки: %1").arg(score));
        // ui->scoreLabel->setText("");
        // ui->hpLabel->setText("");
        update();
        submitScore();
        return;
    }

    update(); // перерисовка
}

void MainWindow::paintEvent(QPaintEvent *)
{
    QPainter painter(this);

```

```

painter.setRenderHint(QPainter::SmoothPixmapTransform, true);

// фон
painter.fillRect(rect(), QColor(125, 125, 125));

// hit-зона
int hitY = height() / 2 - hitZonePixmap.height() / 2;
painter.drawPixmap(hitZoneX, hitY, hitZonePixmap);

if (gameState != GameState::Playing)
    return;

// ВРЕМЯ
qint64 gameTime = gameTimer.elapsed(); // с начала игры
qint64 musicTime = gameTime - preStartDelay; // с начала музыки (может быть < 0)

int startX = hitZoneX + noteTravelDistance;
int y = height() / 2 - notePixmap.height() / 2;

// НОТЫ
for (const Note &note : std::as_const(currentMap.notes)) {

    if (note.hit || note.missed)
        continue;

    qint64 beatTime = note.beatTime;

    // когда нота должна начать ехать (относительно музыки)
    qint64 spawnTime = beatTime - travelTime;

    // сколько она уже едет
    qint64 delta = musicTime - spawnTime;

    // ещё не появилась
    if (delta < 0)
        continue;

    // уже улетела
    if (delta > travelTime + missAfterTime)
        continue;

    double progress = double(delta) / double(travelTime);

    int x = startX - int(noteTravelDistance * progress);

    painter.drawPixmap(x, y, notePixmap);
}

}

void MainWindow::registerHit(qint64 diff)
{
    diff = qAbs(diff);

    int hpDelta = 0;

    if (diff <= 25) {
        lastHitText = "ОТЛИЧНО";
        lastHitColor = QColor(0, 255, 0);
        hpDelta = +2;
        score += 300;
    }
}

```

```

        updateScoreStatusLabel();
    }
    else if (diff <= 50) {
        lastHitText = "ХОРОШО";
        lastHitColor = QColor(0, 200, 255);
        hpDelta = +1;
        score += 200;
        updateScoreStatusLabel();
    }
    else if (diff <= missAfterTime) {
        lastHitText = "СОЙДЕТ";
        lastHitColor = QColor(255, 255, 0);
        hpDelta = 0;
        score += 100;
        updateScoreStatusLabel();
    }
    else {
        lastHitText = "ПЛОХО";
        lastHitColor = QColor(255, 0, 0);
        hpDelta = -5;
    }

    hp += hpDelta;
    updateHpLabel();

    if (hpDelta >= 0)
        hitSound.play();
    else
        missSound.play();

    ui->scoreLabel->setText(lastHitText);
    ui->scoreLabel->setStyleSheet(
        QString("color: %1").arg(lastHitColor.name())
    );

    animateScoreLabel();

    // ПРОИГРЫШ
    if (hp <= 0) {
        gameState = GameState::Idle;
        player->stop();
        ui->statusGameLabel->setText(QString("Вы проиграли. Очки: %1").arg(score));
        update();
    }
}

void MainWindow::animateScoreLabel()
{
    scoreAnim->stop();

    scoreAnim->setStartValue(scoreLabelBasePos + QPoint(0, 10));
    scoreAnim->setEndValue(scoreLabelBasePos);

    scoreAnim->start();
}

void MainWindow::resizeEvent(QResizeEvent *)
{
    // X — центр hit-зоны
    int hitCenterX = hitZoneX + hitZonePixmap.width() / 2;

    // позиция scoreLabel

```

```

int x = hitCenterX - ui->scoreLabel->width() / 2;

int hitY = height() / 2 - hitZonePixmap.height() / 2;
int y = hitY - ui->scoreLabel->height() - 20; // отступ сверху

ui->scoreLabel->move(x, y);

// сохраняем базовую позицию для анимации
scoreLabelBasePos = ui->scoreLabel->pos();

hitCenterX = hitZoneX + hitZonePixmap.width() / 2;
hitY = height() / 2 - hitZonePixmap.height() / 2;

x = hitCenterX - ui->hpLabel->width() / 2;
y = hitY + hitZonePixmap.height() - 20;

ui->hpLabel->move(x, y);
}

void MainWindow::updateHpLabel()
{
    hp = qBound(0, hp, 100);

    ui->hpLabel->setText(QString("HP: %1").arg(hp));

    QColor color;
    if (hp > 60)
        color = QColor(0, 220, 0);
    else if (hp > 30)
        color = QColor(255, 200, 0);
    else
        color = QColor(255, 0, 0);

    ui->hpLabel->setStyleSheet(
        QString("color: %1; font-weight: bold;").arg(color.name())
    );
}

void MainWindow::updateScoreStatusLabel()
{
    if (!scoreStatusLabel)
        return;

    scoreStatusLabel->setText(QString("Очки: %1").arg(score));
}

void MainWindow::showLeaderboard()
{
    if (currentMap.id.isEmpty()) {
        QMessageBox::information(this, "Лидеры", "Сначала загрузите карту.");
        return;
    }

    ui->statusGameLabel->setText("Загрузка таблицы лидеров...");
    leaderboardClient->fetch(currentMap.id);
}

void MainWindow::submitScore()
{
    if (scoreDialogShown || currentMap.id.isEmpty() || hp <= 0)
        return;
}

```

```

scoreDialogShown = true;

bool ok = false;
const QString nickname = QInputDialog::getText(
    this,
    "Сохранить результат",
    QString("Ваш результат: %1 очков\nВведите никнейм:").arg(score),
    QLineEdit::Normal,
    QString(),
    &ok
).trimmed();

if (!ok || nickname.isEmpty())
    return;

leaderboardClient->submit(currentMap.id, nickname, score);
}

void MainWindow::showLeaderboardEntries(const QList<LeaderboardEntry> &entries)
{
    QDialog dialog(this);
    dialog.setWindowTitle("Лидеры: " + currentMap.title);
    dialog.resize(420, 360);

    QVBoxLayout *layout = new QVBoxLayout(&dialog);
    QTableWidgetItem *table = new QTableWidgetItem(entries.size(), 3, &dialog);
    table->setHorizontalHeaderLabels({"Место", "Никнейм", "Очки"});
    table->verticalHeader()->setVisible(false);
    table->setEditTriggers(QAbstractItemView::NoEditTriggers);
    table->setSelectionBehavior(QAbstractItemView::SelectRows);
    table->horizontalHeader()->setSectionResizeMode(QHeaderView::Stretch);

    for (int row = 0; row < entries.size(); ++row) {
        table->setItem(row, 0, new QTableWidgetItem(QString::number(row + 1)));
        table->setItem(row, 1, new QTableWidgetItem(entries[row].nickname));
        table->setItem(row, 2, new QTableWidgetItem(QString::number(entries[row].score)));
    }

    layout->addWidget(table);

    QDialogButtonBox *buttons = new QDialogButtonBox(QDialogButtonBox::Ok, &dialog);
    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    layout->addWidget(buttons);

    ui->statusGameLabel->setText("Таблица лидеров загружена");
    dialog.exec();
}

void MainWindow::onLeaderboardSubmitFinished(bool saved, const QString &message)
{
    if (saved) {
        QMessageBox::information(this, "Лидеры", message);
        showLeaderboard();
    } else {
        QMessageBox::warning(this, "Лидеры", message);
    }
}

void MainWindow::onLeaderboardRequestFailed(const QString &message)
{
    ui->statusGameLabel->setText("Ошибка таблицы лидеров");
    QMessageBox::warning(this, "Лидеры", message);
}

```

```

}

void MainWindow::showSettingsDialog()
{
    if (!settingsDialog) {
        settingsDialog = new SettingsDialog(this);
        connect(settingsDialog, &SettingsDialog::settingsChanged,
            this, &MainWindow::applySettings);
    }

    // Устанавливаем текущие значения
    settingsDialog->setHitVolume(config.hitVolume());
    settingsDialog->setMissVolume(config.missVolume());
    settingsDialog->setMusicVolume(config.musicVolume());
    settingsDialog->setPrimaryKey(config.primaryKey());
    settingsDialog->setSecondaryKey(config.secondaryKey());

    settingsDialog->exec();
}

void MainWindow::applySettings()
{
    if (!settingsDialog)
        return;

    // Сохраняем в конфиг
    config.setHitVolume(settingsDialog->hitVolume());
    config.setMissVolume(settingsDialog->missVolume());
    config.setMusicVolume(settingsDialog->musicVolume());
    config.setPrimaryKey(settingsDialog->primaryKey());
    config.setSecondaryKey(settingsDialog->secondaryKey());

    // Применяем в игре
    hitSound.setVolume(config.hitVolume());
    missSound.setVolume(config.missVolume());
    audioOutput->setVolume(config.musicVolume());

    // Обновляем клавиши
    primaryKey = config.primaryKey();
    secondaryKey = config.secondaryKey();

    // Сохраняем настройки
    config.saveSettings();
}

```

mainwindow.h

```

#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QMediaPlayer>
#include <QAudioOutput>
#include <QElapsedTimer>
#include <QTimer>
#include <QKeyEvent>
#include <QSoundEffect>
#include <QPropertyAnimation>
#include <QLabel>

// Добавляем новый заголовок
#include "leaderboardclient.h"
#include "settingsdialog.h"
#include "gameconfig.h"

```

```

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

protected:
    void keyPressEvent(QKeyEvent *event) override;
    void paintEvent(QPaintEvent *event) override;

private:
    Ui::MainWindow *ui;
    QMediaPlayer *player;
    QAudioOutput *audioOutput;

    // Добавляем диалог настроек
    SettingsDialog *settingsDialog = nullptr;
    GameConfig &config = GameConfig::instance();

    enum class GameState {
        Idle,
        Playing
    };

    GameState gameState = GameState::Idle;

    struct Note {
        qint64 beatTime; // когда должна быть нажата
        bool hit = false;
        bool missed = false;
    };

    struct BeatMap {
        QString id;
        QString title;
        QString musicPath;
        QVector<Note> notes;
    };

    BeatMap currentMap;

    QElapsedTimer gameTimer;
    const qint64 preStartDelay = 1000;

    QTimer *gameLoopTimer;

    const int hitZoneX = 20;
    const int hitZoneSize = 60;
    const int squareSize = 60;

    QPixmap notePixmap;
    QPixmap hitZonePixmap;

    const qint64 travelTime = 1000;
    const int noteTravelDistance = 600;

```

```

const qint64 missAfterTime = 100;

QString lastHitText = "";
QColor lastHitColor = Qt::white;

QSoundEffect hitSound;
QSoundEffect missSound;

QPropertyAnimation *scoreAnim = nullptr;
QPoint scoreLabelBasePos;
int hp = 50;
int score = 0;
bool scoreDialogShown = false;
QLabel *scoreStatusLabel = nullptr;
LeaderboardClient *leaderboardClient = nullptr;

// Добавляем клавиши
int primaryKey = Qt::Key_Space;
int secondaryKey = Qt::Key_X;

private slots:
void loadMap();
void startGame();
void updateGame();
void registerHit(qint64 diff);
void animateScoreLabel();
void resizeEvent(QResizeEvent *event) override;
void updateHpLabel();
void updateScoreStatusLabel();
void showLeaderboard();
void submitScore();
void showLeaderboardEntries(const QList<LeaderboardEntry> &entries);
void onLeaderboardSubmitFinished(bool saved, const QString &message);
void onLeaderboardRequestFailed(const QString &message);

// Добавляем новый слот для настроек
void showSettingsDialog();
void applySettings();
};

#endif // MAINWINDOW_H

```

gameconfig.cpp

```

#include "gameconfig.h"
#include <QDebug>

GameConfig::GameConfig()
: m_settings(QCoreApplication::applicationDirPath() + "/settings.ini",
             QSettings::IniFormat)
{
    loadSettings();
}

void GameConfig::loadSettings()
{
    m_hitVolume = m_settings.value("Sound/hitVolume", 0.4f).toFloat();
    m_missVolume = m_settings.value("Sound/missVolume", 0.4f).toFloat();
    m_musicVolume = m_settings.value("Sound/musicVolume", 0.5f).toFloat();
    m_primaryKey = m_settings.value("Keys/primaryKey", Qt::Key_Space).toInt();
    m_secondaryKey = m_settings.value("Keys/secondaryKey", Qt::Key_X).toInt();

    qDebug() << "Settings loaded:"
               << "hitVol:" << m_hitVolume

```



```

        << "missVol:" << m_missVolume
        << "musicVol:" << m_musicVolume
        << "primaryKey:" << m_primaryKey
        << "secondaryKey:" << m_secondaryKey;
    }

void GameConfig::saveSettings()
{
    m_settings.setValue("Sound/hitVolume", m_hitVolume);
    m_settings.setValue("Sound/missVolume", m_missVolume);
    m_settings.setValue("Sound/musicVolume", m_musicVolume);
    m_settings.setValue("Keys/primaryKey", m_primaryKey);
    m_settings.setValue("Keys/secondaryKey", m_secondaryKey);

    m_settings.sync();
    qDebug() << "Settings saved";
}

void GameConfig::setHitVolume(float volume)
{
    m_hitVolume = qBound(0.0f, volume, 1.0f);
}

void GameConfig::setMissVolume(float volume)
{
    m_missVolume = qBound(0.0f, volume, 1.0f);
}

void GameConfig::setMusicVolume(float volume)
{
    m_musicVolume = qBound(0.0f, volume, 1.0f);
}

void GameConfig::setPrimaryKey(int key)
{
    m_primaryKey = key;
}

void GameConfig::setSecondaryKey(int key)
{
    m_secondaryKey = key;
}

```

gameconfig.h

```

#ifndef GAMECONFIG_H
#define GAMECONFIG_H

#include <QSettings>
#include <QCoreApplication>

class GameConfig
{
public:
    static GameConfig& instance()
    {
        static GameConfig config;
        return config;
    }

    // Геттеры
    float hitVolume() const { return m_hitVolume; }
    float missVolume() const { return m_missVolume; }
    float musicVolume() const { return m_musicVolume; }

```

```

int primaryKey() const { return m_primaryKey; }
int secondaryKey() const { return m_secondaryKey; }

// Сеттеры
void setHitVolume(float volume);
void setMissVolume(float volume);
void setMusicVolume(float volume);
void setPrimaryKey(int key);
void setSecondaryKey(int key);

// Загрузка и сохранение
void loadSettings();
void saveSettings();

private:
    GameConfig();

    float m_hitVolume = 0.4f;
    float m_missVolume = 0.4f;
    float m_musicVolume = 0.5f;
    int m_primaryKey = Qt::Key_Space;
    int m_secondaryKey = Qt::Key_X;

    QSettings m_settings;
};

#endif // GAMECONFIG_H

```

settingsdialog.cpp

```

#include "settingsdialog.h"
#include "ui_settingsdialog.h"
#include <QKeyEvent>
#include <QDebug>
#include <QPushButton>
#include <QTimer>
#include <QDialogButtonBox>

SettingsDialog::SettingsDialog(QWidget *parent)
    : QDialog(parent)
    , ui(new Ui::SettingsDialog)
{
    ui->setupUi(this);

    // Настройка диапазонов слайдеров
    ui->hitVolumeSlider->setRange(0, 100);
    ui->missVolumeSlider->setRange(0, 100);
    ui->musicVolumeSlider->setRange(0, 100);

    // Важно: отключаем стандартную обработку пробела для кнопок
    ui->primaryKeyButton->setAutoDefault(false);
    ui->primaryKeyButton->setDefault(false);
    ui->secondaryKeyButton->setAutoDefault(false);
    ui->secondaryKeyButton->setDefault(false);

    // Подключение сигналов для кнопок клавиш
    connect(ui->primaryKeyButton, &QPushButton::clicked,
            this, &SettingsDialog::onPrimaryKeyButtonClicked);
    connect(ui->secondaryKeyButton, &QPushButton::clicked,
            this, &SettingsDialog::onSecondaryKeyButtonClicked);

    // Находим кнопки buttonBox и настраиваем
    QPushButton *applyButton = ui->buttonBox->button(QDialogButtonBox::Apply);
    QPushButton *okButton = ui->buttonBox->button(QDialogButtonBox::Ok);
}

```

```

QPushButton *cancelButton = ui->buttonBox->button(QDialogButtonBox::Cancel);

if (applyButton) {
    applyButton->setAutoDefault(false);
    applyButton->setDefault(false);
    connect(applyButton, &QPushButton::clicked, this, [this]() {
        emit settingsChanged();
    });
}

if (okButton) {
    okButton->setAutoDefault(false);
    okButton->setDefault(false);
    connect(okButton, &QPushButton::clicked, this, [this]() {
        emit settingsChanged();
        accept();
    });
}

if (cancelButton) {
    cancelButton->setAutoDefault(false);
    cancelButton->setDefault(false);
    connect(cancelButton, &QPushButton::clicked, this, &QDialog::reject);
}

// Обновляем текст кнопок
updateKeyButtonText(ui->primaryKeyButton, primaryKeyCode);
updateKeyButtonText(ui->secondaryKeyButton, secondaryKeyCode);
}

SettingsDialog::~SettingsDialog()
{
    delete ui;
}

float SettingsDialog::hitVolume() const
{
    return ui->hitVolumeSlider->value() / 100.0f;
}

float SettingsDialog::missVolume() const
{
    return ui->missVolumeSlider->value() / 100.0f;
}

float SettingsDialog::musicVolume() const
{
    return ui->musicVolumeSlider->value() / 100.0f;
}

int SettingsDialog::primaryKey() const
{
    return primaryKeyCode;
}

int SettingsDialog::secondaryKey() const
{
    return secondaryKeyCode;
}

void SettingsDialog::setHitVolume(float volume)
{
    ui->hitVolumeSlider->setValue(static_cast<int>(volume * 100));
}

```

```

}

void SettingsDialog::setMissVolume(float volume)
{
    ui->missVolumeSlider->setValue(static_cast<int>(volume * 100));
}

void SettingsDialog::setMusicVolume(float volume)
{
    ui->musicVolumeSlider->setValue(static_cast<int>(volume * 100));
}

void SettingsDialog::setPrimaryKey(int key)
{
    primaryKeyCode = key;
    updateKeyButtonText(ui->primaryKeyButton, key);
}

void SettingsDialog::setSecondaryKey(int key)
{
    secondaryKeyCode = key;
    updateKeyButtonText(ui->secondaryKeyButton, key);
}

void SettingsDialog::keyPressEvent(QKeyEvent *event)
{
    // Если ждем нажатия клавиши для бинда
    if (isWaitingForKey && activeKeyButton && !event->isAutoRepeat()) {
        int key = event->key();

        // Игнорируем модификаторы как отдельные клавиши
        if (key == Qt::Key_Shift || key == Qt::Key_Control ||
            key == Qt::Key_Alt || key == Qt::Key_Meta ||
            key == Qt::Key_AltGr) {
            return;
        }

        // ОСОБАЯ ОБРАБОТКА ПРОБЕЛА
        if (key == Qt::Key_Space) {
            // Для пробела нужно дополнительное действие
            if (activeKeyButton == ui->primaryKeyButton) {
                primaryKeyCode = Qt::Key_Space;
                updateKeyButtonText(ui->primaryKeyButton, Qt::Key_Space);
            } else if (activeKeyButton == ui->secondaryKeyButton) {
                secondaryKeyCode = Qt::Key_Space;
                updateKeyButtonText(ui->secondaryKeyButton, Qt::Key_Space);
            }

            isWaitingForKey = false;
            activeKeyButton = nullptr;

            // Восстанавливаем фокус после пробела
            QTimer::singleShot(100, this, [this]() {
                setFocus();
            });

            event->accept();
            return;
        }

        // Обработка других клавиш
        if (activeKeyButton == ui->primaryKeyButton) {
            primaryKeyCode = key;

```

```

        updateKeyButtonText(ui->primaryKeyButton, key);
    } else if (activeKeyButton == ui->secondaryKeyButton) {
        secondaryKeyCode = key;
        updateKeyButtonText(ui->secondaryKeyButton, key);
    }

    isWaitingForKey = false;
    activeKeyButton = nullptr;
    event->accept();
    return;
}

// Если не в режиме бинда, обрабатываем Escape для отмены
if (event->key() == Qt::Key_Escape && isWaitingForKey) {
    isWaitingForKey = false;
    if (activeKeyButton == ui->primaryKeyButton) {
        updateKeyButtonText(ui->primaryKeyButton, primaryKeyCode);
    } else if (activeKeyButton == ui->secondaryKeyButton) {
        updateKeyButtonText(ui->secondaryKeyButton, secondaryKeyCode);
    }
    activeKeyButton = nullptr;
    event->accept();
    return;
}

QDialog::keyPressEvent(event);
}

void SettingsDialog::onButtonBoxClicked(QAbstractButton *button)
{
    QDialogButtonBox::StandardButton stdButton = ui->buttonBox->standardButton(button);

    switch (stdButton) {
    case QDialogButtonBox::Ok:
        // Применяем настройки и закрываем
        emit settingsChanged();
        accept();
        break;

    case QDialogButtonBox::Apply:
        // Только применяем настройки, не закрываем
        emit settingsChanged();
        break;

    case QDialogButtonBox::Cancel:
        // Просто закрываем без применения
        reject();
        break;

    default:
        // Обработка других кнопок, если есть
        break;
    }
}

void SettingsDialog::onPrimaryKeyButtonClicked()
{
    isWaitingForKey = true;
    activeKeyButton = ui->primaryKeyButton;
    ui->primaryKeyButton->setText("Нажмите клавишу...");
    ui->primaryKeyButton->setFocus();
}

```

```

// Временно меняем политику фокуса для захвата пробела
ui->primaryKeyButton->setFocusPolicy(Qt::StrongFocus);
grabKeyboard(); // Захватываем все события клавиатуры
}

void SettingsDialog::onSecondaryKeyButtonClicked()
{
    isWaitingForKey = true;
    activeKeyButton = ui->secondaryKeyButton;
    ui->secondaryKeyButton->setText("Нажмите клавишу...");
    ui->secondaryKeyButton->setFocus();

    ui->secondaryKeyButton->setFocusPolicy(Qt::StrongFocus);
    grabKeyboard(); // Захватываем все события клавиатуры
}

void SettingsDialog::updateKeyButtonText(QPushButton *button, int key)
{
    button->setText(keyToString(key));

    // Восстанавливаем нормальное состояние кнопки
    if (button == activeKeyButton) {
        releaseKeyboard(); // Освобождаем захват клавиатуры
        activeKeyButton = nullptr;
        isWaitingForKey = false;
    }

    // Восстанавливаем нормальную политику фокуса
    button->setFocusPolicy(Qt::TabFocus);
}

QString SettingsDialog::keyToString(int key)
{
    switch (key) {
        case Qt::Key_Space: return "Пробел";
        case Qt::Key_X: return "X";
        case Qt::Key_Z: return "Z";
        case Qt::Key_A: return "A";
        case Qt::Key_S: return "S";
        case Qt::Key_D: return "D";
        case Qt::Key_F: return "F";
        case Qt::Key_J: return "J";
        case Qt::Key_K: return "K";
        case Qt::Key_L: return "L";
        case Qt::Key_Semicolon: return ";";
        case Qt::Key_Enter:
        case Qt::Key_Return: return "Enter";
        case Qt::Key_Up: return "Стрелка вверх";
        case Qt::Key_Down: return "Стрелка вниз";
        case Qt::Key_Left: return "Стрелка влево";
        case Qt::Key_Right: return "Стрелка вправо";
        case Qt::Key_Tab: return "Tab";
        case Qt::Key_Escape: return "Esc";
        case Qt::Key_Backspace: return "Backspace";
        case Qt::Key_CapsLock: return "Caps Lock";
        case Qt::Key_Delete: return "Delete";
        case Qt::Key_Home: return "Home";
        case Qt::Key_End: return "End";
        case Qt::Key_PageUp: return "Page Up";
        case Qt::Key_PageDown: return "Page Down";
        default:
            QString text = QKeySequence(key).toString();

```

```

        if (text.isEmpty()) {
            return QString("Key %1").arg(key);
        }
        return text;
    }
}

```

settingsdialog.h

```

#ifndef SETTINGSDIALOG_H
#define SETTINGSDIALOG_H

#include <QDialog>
#include <QKeySequenceEdit>
#include <qabstractbutton.h>

namespace Ui {
class SettingsDialog;
}

class SettingsDialog : public QDialog
{
    Q_OBJECT

public:
    explicit SettingsDialog(QWidget *parent = nullptr);
    ~SettingsDialog();

    float hitVolume() const;
    float missVolume() const;
    float musicVolume() const;
    int primaryKey() const;
    int secondaryKey() const;

    void setHitVolume(float volume);
    void setMissVolume(float volume);
    void setMusicVolume(float volume);
    void setPrimaryKey(int key);
    void setSecondaryKey(int key);

signals:
    void settingsChanged();

protected:
    void keyPressEvent(QKeyEvent *event) override;

private slots:
    void onPrimaryKeyButtonClicked();
    void onSecondaryKeyButtonClicked();

    // Убираем старые слоты, добавляем новый
    void onIconButtonClicked(QAbstractButton *button);

private:
    Ui::SettingsDialog *ui;
    bool isWaitingForKey = false;
    QPushButton *activeKeyButton = nullptr;

    int primaryKeyCode = Qt::Key_Space;
    int secondaryKeyCode = Qt::Key_X;

    void updateKeyButtonText(QPushButton *button, int key);
    QString keyToString(int key);
};

```

```
#endif // SETTINGSDIALOG_H
```

leaderboardclient.cpp

```
#include "leaderboardclient.h"
```

```
#include <QJsonDocument>
#include <QJsonObject>
#include <QNetworkReply>
#include <QNetworkRequest>
#include <QUrl>
```

```
#include <algorithm>
```

```
namespace {
const QString LeaderboardBaseUrl = QStringLiteral("https://arcadeleaderboard-5c0d2-default-rtdb.europe-
west1.firebaseio.com");
}
```

```
LeaderboardClient::LeaderboardClient(QObject *parent)
: QObject(parent)
{
}
```

```
void LeaderboardClient::fetch(const QString &mapId)
{
    if (LeaderboardBaseUrl.contains("YOUR-PROJECT")) {
        emit requestFailed("Укажите адрес Firebase Realtime Database в leaderboardclient.cpp.");
        return;
    }
```

```
    QNetworkReply *reply = network.get(QNetworkRequest(mapUrl(mapId)));
    connect(reply, &QNetworkReply::finished, this, [this, reply]() {
        reply->deleteLater();

        if (reply->error() != QNetworkReply::NoError) {
            emit requestFailed("Не удалось загрузить таблицу лидеров: " + reply->errorString());
            return;
        }
```

```
        emit fetchFinished(parseEntries(reply->readAll()));
    });
}
```

```
void LeaderboardClient::submit(const QString &mapId, const QString &nickname, int score)
{
    if (LeaderboardBaseUrl.contains("YOUR-PROJECT")) {
        emit requestFailed("Укажите адрес Firebase Realtime Database в leaderboardclient.cpp.");
        return;
    }
```

```
    QNetworkReply *getReply = network.get(QNetworkRequest(playerUrl(mapId, nickname)));
    connect(getReply, &QNetworkReply::finished, this, [this, getReply, mapId, nickname, score]() {
        getReply->deleteLater();
```

```
        if (getReply->error() != QNetworkReply::NoError) {
            emit requestFailed("Не удалось проверить никнейм: " + getReply->errorString());
            return;
        }
```

```
        const QByteArray currentData = getReply->readAll();
        const QJsonDocument currentDoc = QJsonDocument::fromJson(currentData);
        if (currentDoc.isObject()) {
```



```

        const int oldScore = currentDoc.object().value("score").toInt();
        if (score <= oldScore) {
            emit submitFinished(false, QString("Никнейм уже есть. Его рекорд: %1 очков.").arg(oldScore));
            return;
        }
    }

    QJsonObject record;
    record.insert("nickname", nickname);
    record.insert("score", score);
    record.insert("updatedAt", QDateTime::currentDateTimeUtc().toString(Qt::ISODate));

    QNetworkRequest request(playerUrl(mapId, nickname));
    request.setHeader(QNetworkRequest::ContentTypeHeader, "application/json");

    QNetworkReply *putReply = network.put(request,
    QJsonDocument(record).toJson(QJsonDocument::Compact));
    connect(putReply, &QNetworkReply::finished, this, [this, putReply]() {
        putReply->deleteLater();

        if (putReply->error() != QNetworkReply::NoError) {
            emit requestFailed("Не удалось сохранить результат: " + putReply->errorString());
            return;
        }

        emit submitFinished(true, "Результат сохранен в таблице лидеров.");
    });
}

QUrl LeaderboardClient::mapUrl(const QString &mapId) const
{
    return QUrl(LeaderboardBaseUrl + "/leaderboards/" + firebaseKey(mapId) + ".json");
}

QUrl LeaderboardClient::playerUrl(const QString &mapId, const QString &nickname) const
{
    return QUrl(LeaderboardBaseUrl + "/leaderboards/" + firebaseKey(mapId) + "/" + firebaseKey(nickname) + ".json");
}

QString LeaderboardClient::firebaseKey(QString value) const
{
    value = value.trimmed();
    if (value.isEmpty())
        value = "unknown";

    for (QChar &ch : value) {
        if (ch == '.' || ch == '#' || ch == '$' || ch == '[' || ch == ']' || ch == '/')
            ch = '_';
    }

    return QString::fromLatin1(QUrl::toPercentEncoding(value));
}

QList<LeaderboardEntry> LeaderboardClient::parseEntries(const QByteArray &data) const
{
    QList<LeaderboardEntry> entries;
    const QJsonDocument doc = QJsonDocument::fromJson(data);
    if (!doc.isObject())
        return entries;

    const QJsonObject root = doc.object();

```

```

for (auto it = root.constBegin(); it != root.constEnd(); ++it) {
    if (!it.value().isObject())
        continue;

    const QJsonObject item = it.value().toObject();
    LeaderboardEntry entry;
    entry.nickname = item.value("nickname").toString(it.key());
    entry.score = item.value("score").toInt();
    entry.updatedAt = QDateTime::fromString(item.value("updatedAt").toString(), Qt::ISODate);
    entries.append(entry);
}

std::sort(entries.begin(), entries.end(), [](const LeaderboardEntry &left, const LeaderboardEntry &right) {
    if (left.score == right.score)
        return left.updatedAt > right.updatedAt;
    return left.score > right.score;
});

return entries;
}

```

leaderboardclient.h

```

#ifndef LEADERBOARDCLIENT_H
#define LEADERBOARDCLIENT_H

#include <QObject>
#include <QDateTime>
#include <QList>
#include <QNetworkAccessManager>
#include <QUrl>

struct LeaderboardEntry
{
    QString nickname;
    int score = 0;
    QDateTime updatedAt;
};

class LeaderboardClient : public QObject
{
    Q_OBJECT

public:
    explicit LeaderboardClient(QObject *parent = nullptr);

    void fetch(const QString &mapId);
    void submit(const QString &mapId, const QString &nickname, int score);

signals:
    void fetchFinished(const QList<LeaderboardEntry> &entries);
    void submitFinished(bool saved, const QString &message);
    void requestFailed(const QString &message);

private:
    QUrl mapUrl(const QString &mapId) const;
    QUrl playerUrl(const QString &mapId, const QString &nickname) const;
    QString firebaseKey(QString value) const;
    QList<LeaderboardEntry> parseEntries(const QByteArray &data) const;

    QNetworkAccessManager network;
};

#endif // LEADERBOARDCLIENT_H

```

ВЕДОМОСТЬ

Обозначение					Наименование					Дополнительные сведения		
					<u>Текстовые документы</u>							
БГУИР КР 6-05-0612-01 106 ПЗ					Пояснительная записка					61 с.		
					<u>Графические документы</u>							
ГУИР 451001 106 СП					Игровое программное средство ритмического взаимодействия с сетевым рейтингом.					Формат А1		
										</		